Security Relevant Changes in Reused Agent Skills

SWE 380 / CSC 580 Group 8 Project Report

Jim Prantzalos Kyle Cantrell Vaisnavii Mohanraj Christian Heiney

Fall 2026

Abstract

Agent skills can be copied and adapted while retaining familiar descriptions, even when their instructions change. The group investigated whether modified or reused skills introduce security-sensitive behavior absent from an earlier or source artifact. The pipeline groups GitSkills occurrences into candidate families, identifies related content, records evidence for comparison direction, and evaluates changed static evidence against eleven defined risk indicators. These indicators cover specific changes involving executable downloads, network destinations, dependencies, destructive operations, credentials, permissions, sensitive paths, local scripts, and external instructions. They are review signals rather than confirmed vulnerabilities.

The saved exploratory run contains 1,151 artifacts from 30 provisional families and 3,997 directed comparisons. The assessor reports at least one supported indicator in 588 comparisons (14.7%); 206 comparisons have incomplete evidence coverage. Local script execution, network expansion, and credential-related changes are the most frequently reported categories. Additional graph edges account for most comparisons, and repeated artifacts make observations dependent. The selected families are not a representative sample of GitSkills. Independent review and adjudication for this run remain pending, so these counts measure automated findings rather than validated security incidents or scanner accuracy. The study provides evidence of candidate security-sensitive changes and an auditable way to examine them, while leaving their validated frequency and historical lineage unresolved.

Introduction

Adopting a familiar agent skill can introduce unfamiliar instructions. A reused file may preserve its name and most of its text while adding a downloader, changing a remote code source, or expanding a file operation. Reviewing only the skill description, the presence of a capability, or the total number of scanner matches can miss the change that matters to the adopter.

The Agent Skills format combines a Markdown instruction file with optional scripts, references, and assets (Agent Skills, n.d.). That structure makes the file body and its associated resources relevant to a review. A script can change without changing the instruction that invokes it. Conversely, a new example or additional documentation can increase matches without introducing a new operational requirement. The security question is therefore comparative and contextual: what changed between related artifacts, and does the change satisfy an explicit reason for review?

The study addresses that question through static analysis of the GitSkills dataset (Destefanis et al. 2026). The pipeline separates three decisions: whether two artifacts are meaningfully related, whether an earlier/source-to-later ordering is defensible, and whether changed evidence supports one of eleven risk indicators. A URL reference, a rule-count increase, and a supported risk indicator are distinct observations. None alone establishes malicious intent or execution.

The report presents the comparison method, eleven indicators, exploratory results, and validation requirements. Appendices retain preparation queries, implementation details, validation materials, extended results, and supporting repository resources.

Research question and study scope

> **Do modified or reused skills introduce command execution, file-system access, network access, or other risky behavior that was absent from an earlier version or source artifact?**

The study operationalizes this question as a comparison of stored instructions between related artifacts. An **artifact occurrence** is one skill at a repository and path in the dataset. A **skill variant** groups identical file content, while a **bundle variant** also accounts for the known state of sibling resources. A **candidate family** contains qualifying occurrences with the same exact name. Sharing a name defines a search space; it is not evidence of reuse by itself.

The unit of directional analysis is an ordered pair of related artifacts. Its endpoints are designated **base** and **derived** according to the available direction evidence. These terms identify the comparison orientation, not proven original authorship or direct copying. Equivalent and direction-unknown relationships are retained separately rather than treated as negative security findings.

The primary outcome is whether a comparison supports at least one of the eleven defined risk indicators. The indicator assesses a particular introduced or changed operation, not merely a new keyword, a larger file, or a higher count of matches. For example, changing the source of an executed remote script can matter even if both artifacts contain exactly one download-and-execute command. Replacing a package manager for an otherwise equivalent installation need not introduce a new dependency-acquisition operation.

The group's expectation is that some related artifacts will contain observable security-sensitive changes, but that many differences will reflect routine adaptation, packaging, or legitimate functionality. The competing explanation is that a reported indicator identifies an intended feature rather than unwanted exposure. The study does not measure a causal effect of reuse, execute instructions, or attempt comprehensive security detection. The eleven indicators define the scope of the assessment; behavior outside that scope is not automatically a scanner error.

Background and related work

GitSkills supplies the empirical basis for studying skill occurrences and their copies. Its release retains repository/path identities alongside content hashes, with enrichment for representative content (Destefanis et al. 2026). That distinction motivates the separation of occurrence-level comparisons from distinct content and bundle states. Many comparisons can reuse the same content, so comparison counts cannot be interpreted as independent historical modification events.

The Agent Skills specification defines the instruction file and optional resource organization (Agent Skills, n.d.). It describes a format; it does not establish that a particular skill is trustworthy or that its instructions executed. The assessment uses this structure to inspect available bodies and associated files without execution.

The project's specific contribution is the combination of relatedness, recorded direction evidence, construct-level comparison, and an explicit indicator assessment. The supply-chain concern is the opportunity for changed instructions or dependencies to enter an adopted artifact. The method tests for bounded static signals of that change. It does not classify all artifacts as safe or unsafe, reconstruct complete copying histories, or replace runtime investigation. The scope also distinguishes the present results from the earlier Sprint 1 integration checks, which primarily exercised rule-count comparisons.

Dataset and cohort

Source and candidate construction

The study uses GitSkills version 1.0.0, released on Zenodo on August 10, 2026. The dataset records 3,797,117 skill-file occurrences collected from public GitHub repositories in July 2026 and retains occurrences separately from distinct content (Destefanis et al. 2026). The project imports the released SQLite data into a local DuckDB database for analytical queries. The imported file is a prepared analysis artifact, not the original SQLite download.

Candidate construction requires valid frontmatter, a body longer than 50 characters, non-null content/name/description, and an exact name shared by at least two qualifying occurrences. Artifacts missing any of these attributes were considered unusable to meet the research question's intent.

Description text is normalized for subgrouping, but family analysis loads all qualifying description groups for the chosen name. These filters make comparisons feasible while excluding short, malformed, unnamed, or singleton artifacts that could still contain consequential instructions. They are eligibility criteria, not evidence of safety or copying.

A historical preparation audit on October 15 retained 943,257 candidate occurrences in 162,995 exact-name families. Those audit counts document an earlier local build and are not the denominator for the present cohort results. Appendix A preserves the SQL, cumulative selection counts, and build-local identity limitations. The current saved manifest identifies the cohort inputs and settings; the entire candidate population has not been established as analyzed.

Saved exploratory cohort

The results below describe the saved `sprint2` end-to-end pipeline output, inspected on October 26, 2026. Its manifest labels the selection **provisional**, with no approval reference. Thirty exact-name families were supplied, and the pipeline loaded 1,151 artifacts without skipping an artifact input. Family names span development, scientific, infrastructure, and

security-oriented tasks. Selection was not established as random or representative, so a cohort percentage must not be presented as the prevalence of risk in GitSkills.

The run analyzes all supported primary and additional directed edges within the selected families. Primary edges are the chosen predecessor relationships; additional edges retain other supported comparisons. It records 3,997 analyzed directed pairs and 2,244 ambiguous edges excluded from directional scanning. These are graph-comparison counts rather than counts of unique modifications. Families can differ greatly in size, content repetition, direction evidence, and contribution to the totals.

Evidence availability and handling

Skill bodies are available for the loaded artifacts, but availability of chronology and sibling content varies. A known empty sibling state differs from an unknown or truncated listing. Missing resources cannot be assumed absent historically, and an unmatched rule cannot compensate for unavailable source. The report presents input loading, extraction coverage, and sibling coverage separately because each describes a different boundary.

All dataset content is treated as untrusted text. The pipeline does not execute embedded commands, install referenced dependencies, clone repositories to complete missing records, or retrieve linked instructions. Aggregate tables and schematic examples are sufficient for the report; raw dataset content and credential values are not redistributed. Reproduction depends on the frozen source identities and hashes, not on numeric artifact IDs alone, because IDs can be reassigned in another database build.

Method

Establish relatedness and direction

The relationship stage first separates identical skill content from distinct variants and incorporates known sibling fingerprints into bundle states. It masks leading frontmatter, tokenizes the body, and compares sets of distinct contiguous five-token shingles. The saved policy requires maximum directional containment of at least 0.80, Jaccard overlap of at least 0.30, and at least five shared shingles. These are operational thresholds for candidate relatedness, not probabilities of copying. Identical skill text can still belong to different bundle states.

Direction is assessed only after relatedness. The evidence hierarchy prefers resolved declared provenance, followed by direct observed chronology, equivalent-peer chronology, repository-creation boundaries, and asymmetric containment. A peer date can show that an equivalent skill-and-sibling state existed earlier; it is not the selected artifact's own commit date. Repository creation is used as a boundary, not a skill timestamp. A containment difference of at least 0.10 provides the weakest ordering signal when stronger evidence is absent.

The graph retains one selected predecessor per target while avoiding cycles, and records other supported directions as additional edges. Equivalent or unresolved relationships remain separately visible. The analysis retains the direction basis throughout reporting

because a structural expansion hypothesis warrants a different interpretation from dated or resolved-source evidence. Appendix B contains the rule definitions, run settings, and component diagrams.

Extract and compare observable constructs

The static scanner has thirteen rules across command execution, network access, file-system access, credential access, external code, and system modification. It records source constructs such as commands, URLs, paths, and API calls, with locations and context. Leading skill frontmatter is masked for behavioral scanning while selected provenance fields remain available to relationship analysis. Available sibling file content is considered for applicable bundle changes; directories themselves are not executable evidence.

The comparator preserves added, removed, and retained evidence rather than relying only on match counts. Equal counts can conceal a changed script URL or destination; a larger count can simply reflect repetition. Corresponding operations are associated only when sufficiently clear. Ambiguous correspondences and partial extraction remain uncertain. Local script comparisons use available sibling hashes to distinguish a content change from an identical-content rename.

A separate assessor interprets this comparison using the eleven indicators below. Detection and assessment are intentionally separate: recognizing a package-install command can be correct even when treating the change as newly introduced acquisition is incorrect. Rule counts and capability categories remain diagnostic outputs; they are not substitutes for indicator judgments.

Eleven risk indicators

The following table states the operational scope established in RDR-010. Each indicator requires an observable change and source context. It is a reason for review, not a confirmed vulnerability. The study is limited to these eleven definitions; it does not promise recognition of every security-relevant behavior.

ID	Indicator	Required change
RI-001	Download and execute	Newly downloaded content passes directly to an interpreter.
RI-002	Executed code source	The URL of remotely executed content changes, including its path.
RI-003	Network expansion or redirection	An operation adds a hostname/domain or explicit outbound data transfer.
RI-004	Weakened network security	HTTPS becomes HTTP, or certificate verification is disabled.
RI-005	External code or dependency acquisition	An installation/acquisition operation is introduced, or its source/registry changes. Package-name/version edits alone do not qualify.

ID	Indicator	Required change
RI-006	Destructive or privileged file operation	Destructive operations are introduced or expand in scope or privilege.
RI-007	Credential access or potential exposure	A new operation accesses a recognized credential file or directly includes a recognizable credential reference in an outbound request.
RI-008	System or permissions modification	An operation introduces or expands permissions, service, schedule, or system-configuration changes.
RI-009	Sensitive file-system operation	A read/write operation newly targets a recognized sensitive file or directory.
RI-010	Local script execution	A local invocation is introduced, its target has different known content, or an invoked sibling changes. Identical-content renames do not qualify.
RI-011	External instruction dependency	The skill newly directs following external instructions, changes their source, or turns a reference into such a directive.

Context can suppress a finding or make it uncertain. An explicit warning against an operation should not independently support an instruction to perform it. A documentation link does not establish network execution or an instruction dependency. A newly mentioned credential name does not alone satisfy RI-007. Recognized missing content remains a coverage limitation rather than a negative judgment.

Outcomes and denominators

For each pair, the assessor reports supported and uncertain indicator IDs, supporting evidence, a policy version, and coverage information. The indicator count is the number of distinct supported categories; the finding count deduplicates supporting constructs within that comparison. One credential-file read can support RI-007 and RI-009 while representing one changed construct. Neither count is a severity score.

The main descriptive measure is the number of analyzed directed pairs with at least one supported indicator divided by all analyzed directed pairs in the selected cohort. Per-indicator frequencies use the same denominator and can overlap. The analysis also separates primary/additional edges and complete/incomplete coverage. The overall automated percentage includes incomplete pairs and is labelled accordingly; a complete-case calculation is reported as a sensitivity description, not as an accuracy correction.

Pairs can share artifacts or represent related copies of one change. The analysis does not treat these pairs as independent incidents or use a binomial confidence interval to imply population risk. Appendix B summarizes the pipeline settings; Appendix E identifies supporting documentation and result artifacts.

Results

Automated comparison outcomes

The saved run reports at least one supported indicator in **588 of 3,997 directed comparisons (14.7%)**, across 19 of the 30 provisional families. This is an automated result under the implemented indicator policy. It is not the rate of confirmed vulnerabilities, successful attacks, or independently validated modifications. The saved research summary and pair-level CSV are the sources for the tables in this section.

Measure	Saved result
Selected families / artifacts loaded	30 / 1,151
Directed pairs analyzed	3,997
Pairs with changed extracted evidence	1,998 (50.0%)
Pairs with at least one supported RI	588 (14.7%)
Pairs flagged for any reported reason	2,241 (56.1%)
Pairs with incomplete evidence coverage	206 (5.2%)
Ambiguous edges excluded from directional scanning	2,244

The broader flagged total includes evidence and rule observations and must not be substituted for the supported-indicator total. Likewise, the 1,998 pairs with evidence changes include ordinary differences that do not support an RI. The gap between these measures illustrates why a detected difference is insufficient to answer the research question.

Types of reported change

Local script execution (RI-010), network expansion/redirection (RI-003), and credential access/potential exposure (RI-007) occur most often in the automated output. Each row below counts pairs supporting that indicator at least once. A pair may appear in several rows; percentages are not portions of a mutually exclusive distribution.

Indicator	Pairs	Percent of all 3,997 pairs
RI-001 Download and execute	96	2.40%
RI-002 Executed code source changed	0	0.00%
RI-003 Network expanded or redirected	381	9.53%
RI-004 Network security weakened	22	0.55%
RI-005 Code or dependency acquisition	179	4.48%
RI-006 Destructive or privileged file operation	41	1.03%
RI-007 Credential access or potential exposure	231	5.78%
RI-008 System or permissions modification	53	1.33%
RI-009 Sensitive file-system access	0	0.00%
RI-010 Local script execution	413	10.33%
RI-011 External instruction dependency	0	0.00%

Zero supported observations for RI-002, RI-009, and RI-011 do not establish that those changes never occur. Their absence can reflect this cohort, limited evidence, matching logic, uncertain assessments, or the difficulty of associating operations across artifacts. Validation must inspect negative cases before concluding that a zero frequency demonstrates successful absence detection.

The assessor reports 5,551 supporting findings summed across comparisons. These are deduplicated constructs within pairs, not 5,551 globally unique actions. Repeated artifacts and additional edges can count related evidence again. The report uses the figure only to describe report volume, not as a measure of distinct harmful behavior.

Edge selection and direction evidence

Primary edges contribute 468 comparisons, of which 81 have supported indicators (17.3%). Additional edges contribute 3,529 comparisons, of which 507 have indicators (14.4%). Most observations therefore come from comparisons beyond the single selected predecessor. Pooling them without an edge-type breakdown would obscure how the graph contributes to the reported frequency.

Direction is based on direct chronology for 237 pairs, equivalent-peer chronology for 996, repository boundaries for 1,046, containment for 1,713, and resolved source declarations for five. Containment alone accounts for 42.9% of comparisons and 401 of the 588 indicator-bearing pairs. Consequently, many apparent introductions depend on the weakest direction basis. These cases can show meaningful differences even where the historical order remains contestable.

Family contributions are also uneven. One family contributes 1,136 comparisons (28.4% of the cohort) and 298 indicator-bearing pairs (50.7% of all indicator-bearing pairs). This concentration makes the pooled rate sensitive to family selection and graph density. Appendix D provides the complete family and direction-basis tables so readers can inspect this imbalance rather than assume every family contributes equally.

Coverage and complete-case description

Of the 3,997 pairs, 206 have incomplete evidence coverage, including 117 with missing sibling content. The remaining incompleteness can involve partial or unsupported extraction and other recorded evidence issues. The execution summary's 117 incomplete sibling comparisons is therefore narrower than the research summary's 206 pairs with incomplete evidence coverage; these are different fields, not interchangeable totals.

The pair-level table contains 3,791 comparisons labelled complete, with 484 indicator-bearing pairs (12.8%). Another 104 indicator-bearing pairs occur among the 206 incomplete comparisons. Restricting to complete-labelled cases changes the descriptive rate, but cannot correct selection bias or prove that the detector recognized all relevant operations. The main total of 588 includes both groups and should always be accompanied by this coverage distinction.

Representative interpretation cases

Equivalent installation syntax. A reviewed development example replaces `bun add` with `npm install` for the same dependencies and sync workflow. Installation is already present, and no new registry or source is demonstrated. The rule can correctly identify both commands while RI-005 is a false positive for a newly introduced acquisition operation. An explicitly incorrect missing-sync example must not add independent positive support. This is a test of the existing RI definition, not a request to detect additional categories.

A retained credential pattern. A client SDK example adds another initialization block using a framework-exposed authentication-key variable. The base already contains the same key-based initialization pattern. The name may attract review, but its added occurrence alone does not establish introduced RI-007. A separate, newly credential-bearing outbound operation would need its own evidence. This case illustrates why keyword presence and a change in credential behavior must be judged separately.

Download and permission changes. Another example adds remote retrieval followed by an executable-permission change and invocation of the downloaded file. The saved assessor reports network expansion and permission modification. Inspection can ask whether acquisition or deletion also satisfies an existing indicator. However, a download-then-execute sequence is not automatically the direct interpreter pipeline defined by RI-001, and a newly introduced HTTP URL is not automatically RI-004's HTTPS downgrade. Payload format, unavailable content, and direction limitations must remain explicit.

These examples illustrate interpretation issues identified during development and review discussion. They are not a completed, independently adjudicated accuracy sample for the current frozen outputs. Appendix D provides the current cohort tables, while Appendix E identifies the underlying result artifacts.

Evaluation and validation

What is established by the saved run

The pipeline completed the automated workflow for the supplied cohort, loaded all 1,151 selected artifacts, preserved pair-level outcomes, and generated review samples. Completion establishes that those stages produced the saved outputs. It does not establish that relationships are historical facts, that every supported RI is correct, or that unflagged cases lack in-scope changes. The execution summary explicitly states that independent review and adjudication are awaited.

Earlier development checks exercised constructed fixtures, file-input comparisons, extraction, and output round trips. These provide regression evidence for specified behavior, not independent estimates of detector precision or recall. Appendix C summarizes the current validation scope and status. No new scanner test result is claimed from rebuilding this report, and an older passing test count is not presented as validation of the current research run.

Current review design

The saved directed sample contains 30 pairs: five each from supported-indicator, uncertain-indicator, coverage-limited, sibling-change, flagged-without-indicator, and unflagged-directed strata. A separate relationship sample contains 15 pairs: five ambiguous, five equivalent, and five with no related edge. Both use seed 580. Sampling includes negative and difficult cases because reviewing only reported positives cannot establish recall or reveal missed relationships.

For a directed pair, reviewers inspect both original bodies, relevant available siblings, the source correspondence, and direction evidence. They judge each of RI-001 through RI-011 independently of the scanner's predicted IDs. Relatedness, direction, rule matches, evidence changes, supported/uncertain indicators, and coverage receive separate fields. For a relationship pair, reviewers assess substantive relatedness and defensible ordering, including pairs that the automated method did not link.

Independent copies precede team adjudication. The accepted annotations must match the frozen sample's identities and generated values; importing judgments from an earlier family list or regenerated sample without reconciliation would invalidate the link between labels and results. AI-assisted proposals can help identify source locations and questions, but are not an additional independent human reviewer. Acceptance of an individual explanatory example also does not establish completion of the full current sample.

Indicator accuracy and uncertainty

A false positive occurs when the scanner supports an RI that manual review finds unsupported under its defined change conditions. A false negative occurs when manual review supports one of those eleven RIs and the scanner does not report it as supported. Missing an additional command, API, or keyword is not automatically an RI false negative: the behavior may be retained, uncertain, outside the eleven definitions, or additional evidence for an RI already reported.

For example, missing a second network-client syntax does not create another missed RI-003 category if the pair already supports and reports RI-003. Conversely, correctly extracting a package-install rule does not excuse an RI-005 false positive when the operation is merely an equivalent substitution. Separating these judgments keeps evaluation aligned with the research instrument rather than expanding it into an unbounded security checklist.

The reporting protocol includes indicator metrics only for assessable comparisons: manual relatedness must be related, direction supported, scanner and reviewer coverage complete, and all eleven categories reviewed. Uncertain individual indicators are excluded from their corresponding judgments. Other reviewed pairs remain informative cases but do not become negative labels. Precision measures support among reported positives; recall requires independently identified positives, including omissions.

The current saved summary contains no completed manual-validation or relationship-validation metrics. The report therefore provides no precision, recall, or reviewer-agreement estimate. The stratified review sample is designed to test the tool and inspect errors;

without appropriate weighting it is not a prevalence sample. Appendix E links to the detailed protocol and annotation instructions.

Discussion

Answer supported by the present evidence

The automated results are consistent with the presence of security-sensitive changes among related skills: 588 comparisons satisfy at least one implemented indicator, and available examples contain concrete additions such as remote retrieval or permission changes. The supported answer is that the pipeline identifies candidate introductions warranting review within the selected cohort. The validated frequency of such introductions remains unresolved because cohort approval and independent adjudication are not established in the saved run.

This distinction matters for the research question. A new operation can be visible in a supplied comparison even when the reason to call one occurrence historically earlier is weak. The content difference then remains useful, but the claim that modification introduced it must be conditional on direction. Similarly, a legitimate package installation or credential-bearing request can satisfy a review indicator without being malicious. The result should be read as evidence to inspect, not as a causal claim that reuse produces insecurity.

Why the comparison layers matter

The study's most useful methodological distinction is between a rule match, an evidence change, and an RI. Rules identify candidate constructs; comparison identifies what differs; assessment decides whether the difference meets a defined trigger. Each layer answers a separate question. Conflating them can turn routine maintenance into a finding or hide a changed executable source behind unchanged totals.

The installation-substitution example demonstrates the practical consequence. The scanner can recognize both commands correctly, but the assessor still needs to recognize the retained operation and its context. The credential example similarly requires checking the base rather than treating a new occurrence of a key-like name as new exposure. Fixing either issue should target the affected indicator logic or evidence correspondence, not redefine every credential term or executable command as a new risk category.

Competing explanations and implications

The distribution can reflect family selection, graph density, available enrichment, and legitimate differences in function. A scientific or administrative skill may need scripts and system operations; an expanded integration guide may add dependencies as an intended feature. Longer artifacts provide more opportunities for recognized constructs, and one heavily connected family can dominate comparison counts. None of these explanations requires malicious propagation.

For reviewers, the actionable output is a source-backed change with its direction and coverage limitations. For tool development, the priority is faithful implementation and validation of the existing eleven definitions, including equivalent operations, warnings, and unchanged-content controls. A higher finding count is not itself an improvement. A frozen comparison set with independently accepted labels is needed to tell whether a change reduces false positives without introducing false negatives.

The results also motivate separating primary and additional comparisons when communicating supply-chain exposure. Additional edges help inspect possible relationships, but their volume can amplify the same content across multiple pairings. A report should expose that dependence and family concentration instead of interpreting the pooled 14.7% as a probability that adopting an arbitrary skill introduces harmful behavior.

Threats to validity

Construct validity. The eleven RIs are an operational definition of selected security-sensitive changes, not an exhaustive taxonomy of security behavior. Regex matches, recovered constructs, and source-context checks can misinterpret warnings, examples, aliases, or equivalent operations. Complete extraction means a construct was recovered, not that its runtime effect was resolved. Zero supported RIs therefore cannot establish safety, and larger counts cannot establish greater danger.

Relatedness and historical order. Exact-name candidate generation can miss renamed relatives and group independent skills together. Shingle overlap supports substantive similarity but does not prove copying; connected clusters need not have pairwise-related endpoints everywhere. Commit history follows current paths and can omit earlier states. Equivalent-peer dates and repository boundaries are useful evidence but do not establish original authorship. Containment-based direction, used for 42.9% of this run's pairs, is particularly weak for a historical introduction claim.

Missing evidence and scope. Source enrichment is uneven, especially for sibling resources and chronology. Unknown composition is not an empty bundle. The run reports 206 incomplete comparisons, while even complete-labelled coverage cannot guarantee all risky instructions were recognized. Conditional sibling scanning and unavailable implementations can limit conclusions about indirect execution. Coverage must be checked against stored source availability during validation rather than accepted solely from a summary label.

Selection and dependence. The 30-family list is provisional and not a representative sample. Eligibility filters exclude metadata-poor, short, and singleton skills. The 3,997 comparisons share artifacts, and a single family contributes half the indicator-bearing pairs. Primary and additional edges have different selection roles. Consequently, the percentages describe these outputs; they are not population estimates, independent incident rates, or causal effects of modification/reuse. Historical Sprint 1 results use different units and must not be pooled with them.

Validation and reproducibility. Independent adjudication for the current sample is pending, so accuracy and agreement are unknown. AI-assisted draft judgments do not supply an independent human reference standard. Numeric IDs are local to a prepared database build; stable source identity, content hashes, policy, code revision, and sample checksums are needed to reconnect labels to a run. The manifest records fingerprints but the prepared database size alone is not a cryptographic identity check. A regenerated cohort or changed implementation requires checking compatibility before reusing annotations.

These limits are addressed by retaining source evidence, direction bases, uncertain outcomes, and coverage separately, and by providing reproducible inputs and review forms. They are not eliminated by those mechanisms. The strongest remaining requirement is an approved, frozen cohort with independent source-based review and adjudication before numerical accuracy or general prevalence is claimed.

Conclusion

Project Group 8 investigated whether modified or reused agent skills introduce security-sensitive behavior absent from an earlier or source artifact. The implemented method combines candidate relatedness, recorded direction evidence, static construct comparison, and eleven explicit risk indicators. The saved provisional cohort yields supported automated indicators in 588 of 3,997 directed comparisons, with local-script, network, and credential-related changes reported most frequently.

These findings establish a set of candidate changes for review, not 588 confirmed incidents or a population-wide risk rate. Independent adjudication remains pending, coverage is incomplete for some comparisons, and many directions rely on structural rather than temporal evidence. The research question therefore has a qualified empirical answer: observable candidate introductions occur in the analyzed outputs, while their validated frequency and historical interpretation require further evidence.

The next research step is to freeze and approve the cohort, complete both review assignments, adjudicate disagreements, and calculate accuracy only for assessable cases. Tool refinements should first improve fidelity to the existing eleven definitions, including equivalent installation operations and explicit counterexamples. Keeping detection, comparison, and indicator interpretation separate makes the pipeline useful without claiming comprehensive security detection.

References

Agent Skills. n.d. *Specification*. Accessed October 26, 2026. https://agentskills.io/specification.

Destefanis, Giuseppe, Daniel Graziotin, Matteo Vaccargiu, and Marco Ortu. 2026. *GitSkills: A Dataset of Agent Skills on GitHub*. Version 1.0.0. Zenodo. https://doi.org/10.5281/zenodo.21875637.

Appendix A Dataset construction and preparation

Candidate selection

GitSkills is imported from the released SQLite database into a separate DuckDB database. The following query from `sql/create_analysis_tables.sql` materializes the comparison candidates without changing the source `artifacts` table.

```
CREATE TABLE artifact_groupings AS
WITH filtered AS (
    SELECT
        a.id AS artifact_id,
        a.repo_id,
        a.name,
        regexp_replace(
            lower(trim(a.description)),
            '\s+',
            ' ',
            'g'
        ) AS normalized_description
    FROM artifacts AS a
    WHERE a.frontmatter_valid = 1
      AND a.body_chars > 50
      AND a.content IS NOT NULL
      AND a.name IS NOT NULL
      AND a.description IS NOT NULL
),
eligible_names AS (
    SELECT
        name
    FROM filtered
    GROUP BY name
    HAVING COUNT(*) >= 2
),
groups AS (
    SELECT
        f.name,
        f.normalized_description,
        COUNT(*) AS artifact_group_size
    FROM filtered AS f
    JOIN eligible_names AS e ON e.name = f.name
    GROUP BY
        f.name,
        f.normalized_description
),
group_ids AS (
    SELECT
```

```
        row_number() OVER ()::BIGINT AS id,
        name,
        normalized_description,
        artifact_group_size
    FROM groups
)
SELECT
    g.id,
    f.artifact_id,
    f.repo_id,
    f.name,
    f.normalized_description,
    g.artifact_group_size,
    NULL::BIGINT AS sibling_file_count,
    NULL::VARCHAR AS sibling_content_sha
FROM filtered AS f
JOIN eligible_names AS e ON e.name = f.name
JOIN group_ids AS g ON g.name = f.name
 AND g.normalized_description = f.normalized_description;
```

Candidates require valid frontmatter, more than 50 body characters, available content/name/description, and at least two qualifying occurrences with the same exact name. These conditions select analyzable comparison candidates; they do not establish safety, reuse, or common ancestry. Short, malformed, metadata-poor, and singleton skills are excluded.

Descriptions are lowercased, trimmed, and whitespace-normalized for subgrouping. Family analysis compares across description subgroups within the same exact name, preserving candidates whose descriptions changed. The repeated-name requirement counts occurrences, including identical copies, rather than distinct content or repositories.

Preparation workflow

The activity diagram connects candidate preparation to the end-to-end cohort analysis. Preparation counts come from the October 15, 2026 local audit; the final analyzed cohort is a supplied subset of this population.

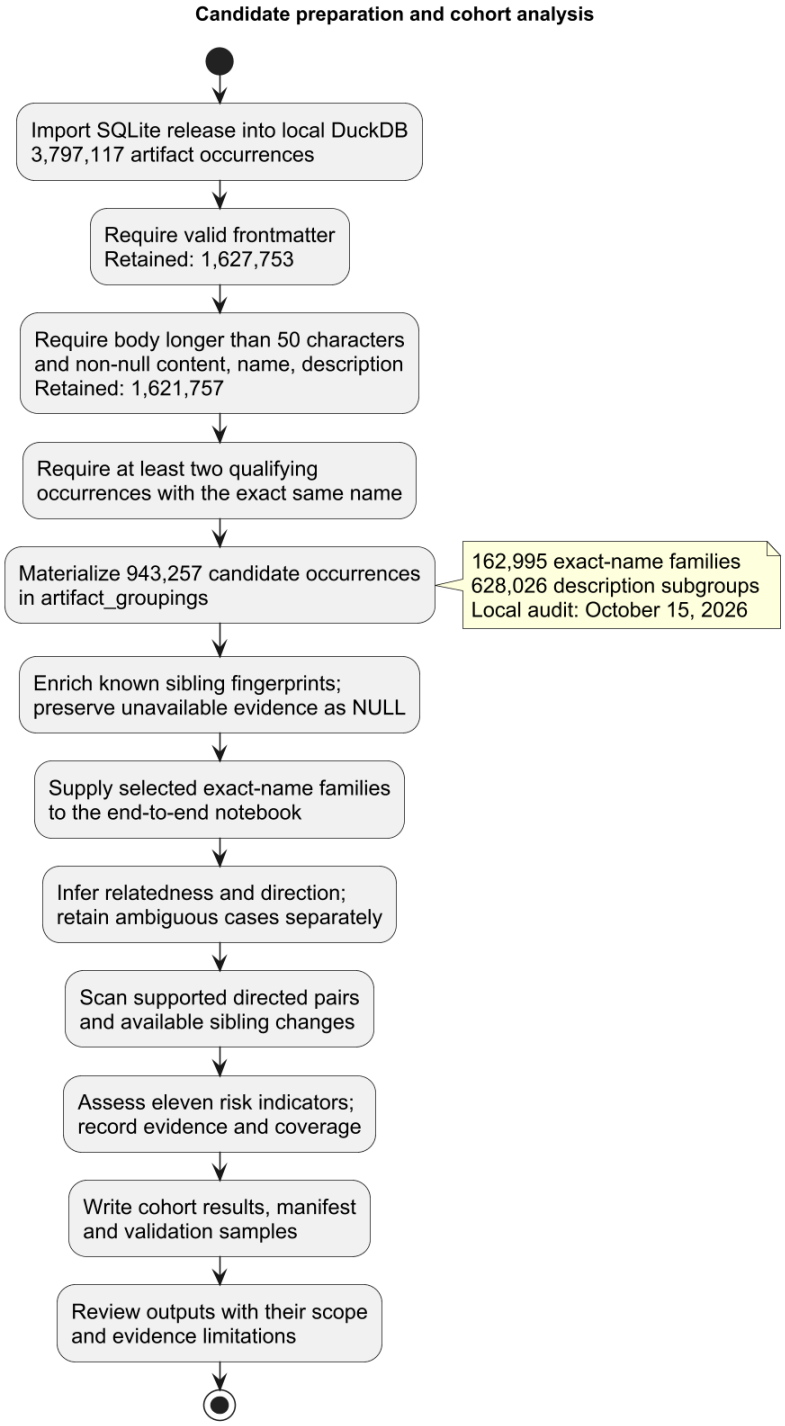


Figure 1: Candidate preparation and the end-to-end cohort analysis workflow.

Selection audit and identity

Cumulative selection stage	Retained artifacts	Excluded at this step
All source artifact occurrences	3,797,117	-
`frontmatter_valid = 1`	1,627,753	2,169,364
`body_chars > 50`	1,621,757	5,996
`content IS NOT NULL`	1,621,757	0
`name IS NOT NULL`	1,621,757	0
`description IS NOT NULL`	1,621,757	0
Exact name occurs in at least two qualifying rows	943,257	678,500

The audit retained 943,257 occurrences in 162,995 exact-name families. These preparation totals are not the results denominator: the saved provisional run analyzes 1,151 artifacts from 30 supplied families. Family selection was not random or established as representative.

Artifact and description-group numbers are local to the prepared database. The query assigns group numbers without an explicit ordering, so cross-build reconciliation must use the dataset snapshot, repository/path identity, and content hashes. The sibling fields begin as null placeholders; subsequent enrichment distinguishes known empty bundles from unavailable or incomplete sibling evidence. Null must not be interpreted as an empty bundle.

Appendix B Pipeline implementation

Rule definitions

The pipeline uses thirteen extraction rules across six categories. These rules identify candidate evidence; the eleven RIs in the main report assess whether a changed operation satisfies a defined indicator. A new match or larger match count alone does not establish an RI.

Category	Registered rules and indicators	Reason for inclusion	Interpretation limit
Command execution	CMD-001: shell-language code fences; CMD-002: recognized command invocations	Instructions can launch interpreters and executable programs.	A fence or command reference can be explanatory documentation.
Network access	NET-001: HTTP/HTTPS URLs; NET-002: recognized network-client usage	Instructions can contact external resources or retrieve data.	A URL alone does not show a request or data transfer.
File-system access	FS-001: file commands; FS-002: writing APIs; FS-003: reading commands/APIs	Instructions can read or modify local data.	Detection does not establish sensitive targets or harmful changes.
Credential access	CRED-001: secret-related terms; CRED-002: credential-file references	Instructions can involve authentication material and secret-bearing resources.	A reference is not evidence that a real secret was exposed.
External code execution	EXT-001: download-to-shell pipelines; EXT-002: package installation	Instructions can introduce externally supplied executable content.	Legitimate dependency installation is common; source trust is not verified.
System modification	SYS-001: privilege elevation; SYS-002: system-changing commands	Instructions can affect permissions and host configuration.	Administrative workflows may legitimately require these commands.

The executable definitions are in `src/gitskills/rules/registry.py` and the category modules. Skill-body scans mask leading frontmatter while preserving source locations; sibling content is scanned separately. Overlapping rules can describe the same instruction and

must not be counted as independent actions. The scanner does not execute the analyzed content.

End-to-end processing

The end-to-end notebook invokes the cohort summarizer directly. It loads the selected exact-name families, establishes relatedness and direction, scans supported directed comparisons, assesses changed constructs against RI-001 through RI-011, and writes cohort summaries, coverage records, and validation samples. Compatible cached scans avoid repeating the same work; a saved manifest connects the outputs to source identities and run settings.

The saved run uses five-token shingles, containment of at least 0.80, Jaccard similarity of at least 0.30, and at least five shared shingles for distinct skill variants. Direction considers resolved source declarations, observed chronology, equivalent-peer chronology, repository boundaries, and finally containment asymmetry with a 0.10 margin. These are inference criteria rather than proof of copying. Ambiguous relationships remain outside directional scanning.

Primary edges select an acyclic predecessor structure; additional edges retain other supported comparisons. Both are included in the reported run. Skill and sibling states remain distinct so a changed script can be assessed even when its invoking skill text is unchanged. Unavailable sibling content is recorded as a coverage gap.

Relationship classification

The state diagram summarizes comparison eligibility and relationship outcomes. These outcomes are computed classifications; they do not represent verified historical descent.

Scanning interactions

The sequence diagram shows the family-scanning component used within the cohort pipeline, including conditional sibling analysis and comparison.

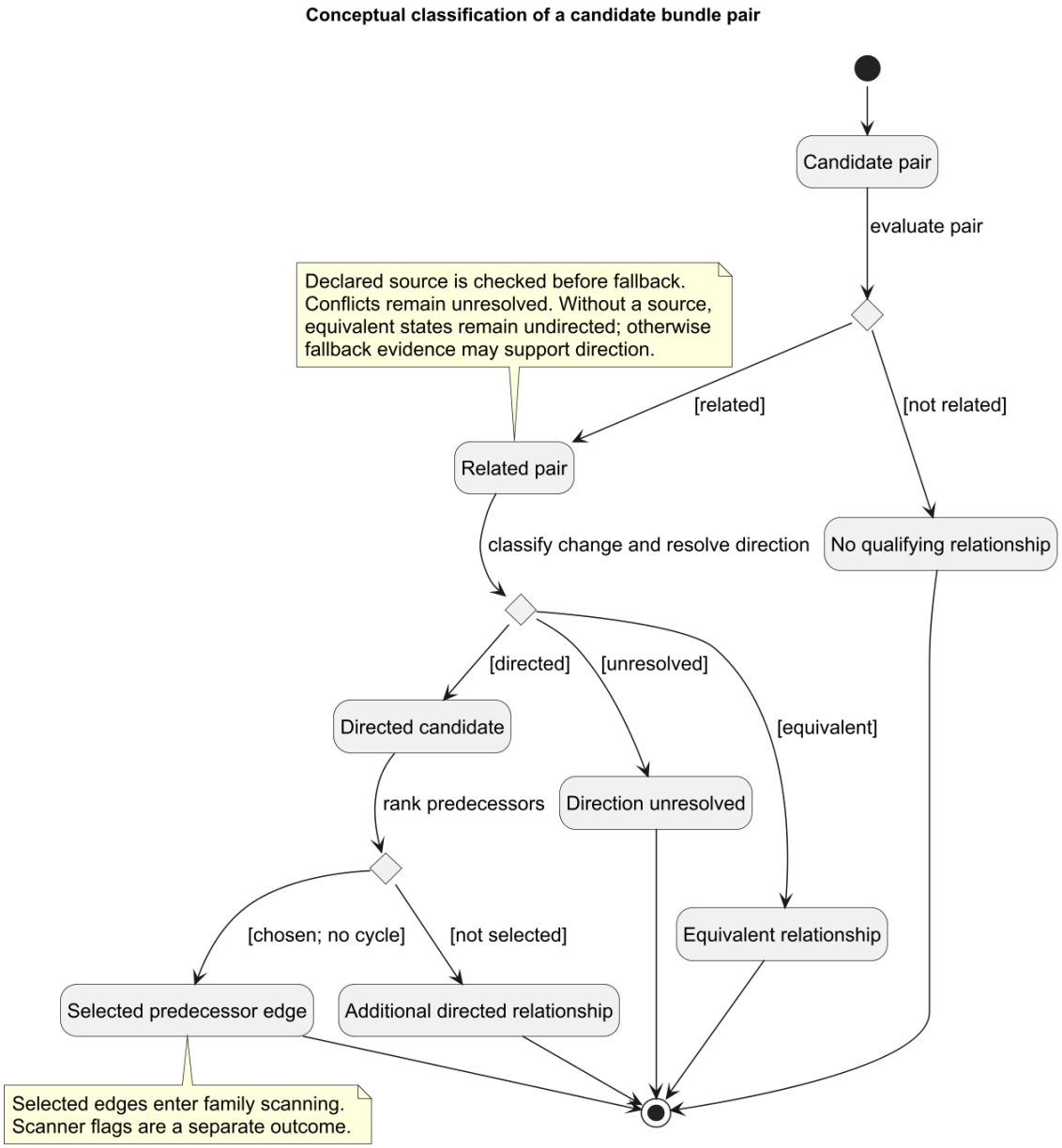


Figure 2: Conceptual UML state diagram for relationship classification. Scanner flags are separate from lineage outcomes.

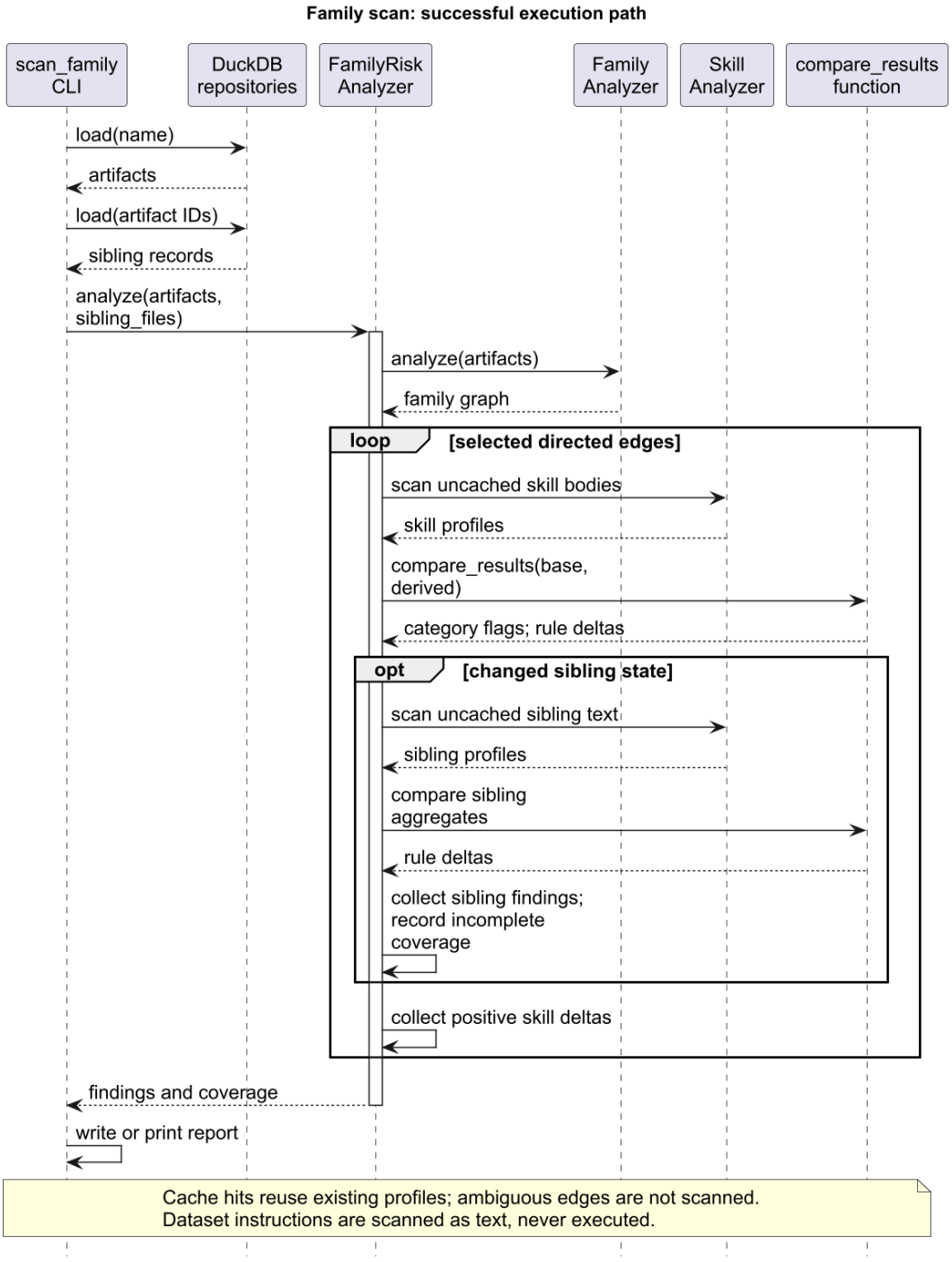


Figure 3: Sequence diagram of family scanning, including conditional sibling analysis and comparison.

Appendix C Validation scope and status

The saved run generated 30 directed review pairs across six strata and 15 relationship review pairs across three strata, using seed 580. The samples include supported, uncertain, coverage-limited, and unflagged cases. Their purpose is to inspect pipeline decisions, including omissions, rather than estimate population prevalence.

Manual judgments distinguish relatedness, direction, extraction rules, evidence changes, and the eleven defined RIs. An RI false positive is an unsupported reported indicator; an RI false negative is a manually supported in-scope indicator absent from the scanner's supported set. Retained behavior, equivalent syntax, and observations outside RI-001 through RI-011 do not automatically constitute missed indicators.

The current saved summary contains no completed independent validation metrics. Precision, recall, and agreement therefore remain unreported. Detailed annotation instructions and field definitions are retained in the repository resources listed in Appendix E rather than reproduced here.

Appendix D Extended cohort results

Current cohort by family

These are saved automated counts for the provisional 30-family run. Families are source-selection labels, not maliciousness classifications. Indicator and incomplete-coverage columns can overlap; Gaps counts pairs with incomplete evidence coverage. No source bodies or credential values are reproduced.

Family	Artifacts	Pairs	With RIs	Gaps
003-create-golang-project	4	1	1	0
1k-defi-module-integration	4	3	1	0
PromptInjection	10	32	5	0
azure-communication-sms-java	33	255	0	0
busybox-on-windows	26	174	0	0
capacitor-best-practices	3	3	2	2
caveman-help	46	363	0	0
cometchat-core	5	7	3	0
cron-scheduler	25	8	0	0
deepread	10	17	0	17
defi-protocol-templates	47	544	58	0
devcontainer-setup	41	249	52	65
exploiting-oauth-misconfiguration	18	49	22	49
fullstack-developer	44	81	11	0
gitlab-api	15	1	1	1
hello-world	171	4	0	1
implementing-disk-encryption-with-bitlocker	17	26	10	0
kotlin-multiplatform	13	4	1	0
legacy-migration-planner	5	9	0	0
modern-best-practice-react-components	10	36	8	0
neurokit2	40	285	55	14
opentargets-database	31	83	7	7
performing-graphql-depth-limit-attack	18	37	0	0
prompt-transformer	10	20	0	0
python-performance-optimization	72	460	9	0
seo	287	1136	298	49
shell-scripting	44	1	0	0
system-info	43	9	4	0
yaml-master	12	5	0	0
youtube-downloader	47	95	40	1

Current cohort by direction basis

Direction basis	Pairs	With RIs	Incomplete coverage
chronology	237	43	33
containment	1713	401	74
declared-source	5	0	0
equivalent-peer-chronology	996	69	58
repo-created-boundary	1046	75	41

Primary and additional indicator counts

The same comparison can support several categories. Totals across rows are therefore not distinct comparisons.

Indicator	Primary pairs	Additional pairs
RI-001	7	89
RI-002	0	0
RI-003	49	332
RI-004	12	10
RI-005	35	144
RI-006	7	34
RI-007	13	218
RI-008	7	46
RI-009	0	0
RI-010	44	369
RI-011	0	0

Appendix E Supporting repository material

The following repository material supports the analysis without being reproduced in the report. Links identify resource locations; the saved run's manifest, rather than the evolving main branch, identifies the analysis inputs and implementation.

Resource	Relevance
End-to-end notebook	Executable cohort analysis and generation of result tables.
Pipeline README	Current setup, configuration, output layout, and validation instructions.
Dataset setup	Source-data acquisition and preparation of the local DuckDB database.
Candidate-selection SQL	Executable selection query reproduced in Appendix A.
Sibling enrichment SQL	Known and unknown sibling-state preparation.
Risk indicator definitions	Operational definitions and scope of RI-001 through RI-011.
Annotation protocol	Detailed review fields and adjudication instructions.
Threat model	Scope and interpretation limits of static security evidence.
Report build guide	Report sources, appendix ordering, and local rendering instructions.

The saved outputs are under `notebooks/end_to_end_pipeline/results/sprint2/`. The sample manifest records source identities, configuration, and sample checksums. Within `review/`, the summary, directed-pair, family, indicator-frequency, and direction-basis files supply the main results and Appendix D tables. The execution summary and audit files record processing status and coverage. Annotation files remain separate from automated predictions.

Appendix F Use of AI in development and report preparation

Project Group 8 used AI assistance during tool and pipeline development and in preparing this report. Development assistance included discussing implementation approaches, drafting and revising code, troubleshooting errors, and supporting documentation and verification work. These activities contributed to the components integrated into the end-to-end pipeline.

For the report, AI assistance supported organizing material, drafting and revising prose, condensing earlier documentation, and checking consistency between the narrative and saved outputs. It also supported formatting and rebuilding the report. The numerical findings come from the saved pipeline artifacts; AI-generated explanations are not experimental measurements.

Project Group 8 remains responsible for the implementation, interpretation, and final submitted text. Generated code and prose require checking against project requirements, source evidence, and reproducible outputs. AI assistance does not establish the correctness of inferred relationships or reported indicators, and it does not resolve the validation limitations stated in the main report.